ADVANCED SYSTEMS LAB · TEAM 41 · FINAL

Interval-Masked Streaming Top-k Pearson Correlation

Recap on memory hierarchy optimizations
+ memory *layout* sorting optimization

$0.06 \rightarrow \approx 10$ FLOPs/cyc (`LONG`) ≈ 10 FLOPs/cyc (`SHORT`) 9 incremental improvements

Recap

Matrix $X \in \mathbb{R}^{F \times S}$ — each feature i valid only on $[\text{start}_i, \text{end}_i)$.

- For every feature, find the $K = 16$ others with the largest $\rho_{I_i[\ldots],\, I_j[\ldots]}$
- Over overlap $L = \max(\text{start}_i, \text{start}_j),\ R = \min(\text{end}_i, \text{end}_j)$
- $\text{corr}(i,j) = \dfrac{1}{R - L - 1} \sum_{t=L}^{R-1} z_i(t)\, z_j(t)$
- R1: never materialize the full $F \times F$ matrix → streaming top-K heap

3 phases, 1 bottleneck

Phase 1: Normalize

μ_i, σ_i per feature $\rightarrow$ z-scores, once

3.4% of cycles

Phase 2: Correlate

z-score dot product over every overlap $[L, R)$

96.6% of cycles

Phase 3: Top-K

min-heap insert per pair

0.1% of cycles

```
// Phase 2 inner loop
float acc = 0.0f;
for (uint32_t t = L; t < R; ++t)
    acc += Z[i*S + t] * Z[j*S + t];
corr = acc / (R - L - 1);
```

Microbenchmark (PAPI, $F{=}128$, $S{=}8192$):
Phase 2 = **96.6%** of cycles

Optimizations all focus on this dot product.

Measurement: growing warm-up batches prime branch predictors & reach steady state, then every input buffer + scratch is `CLFLUSH` + `MFENCE` 'd before each timed call — **numbers start from a cold cache**.

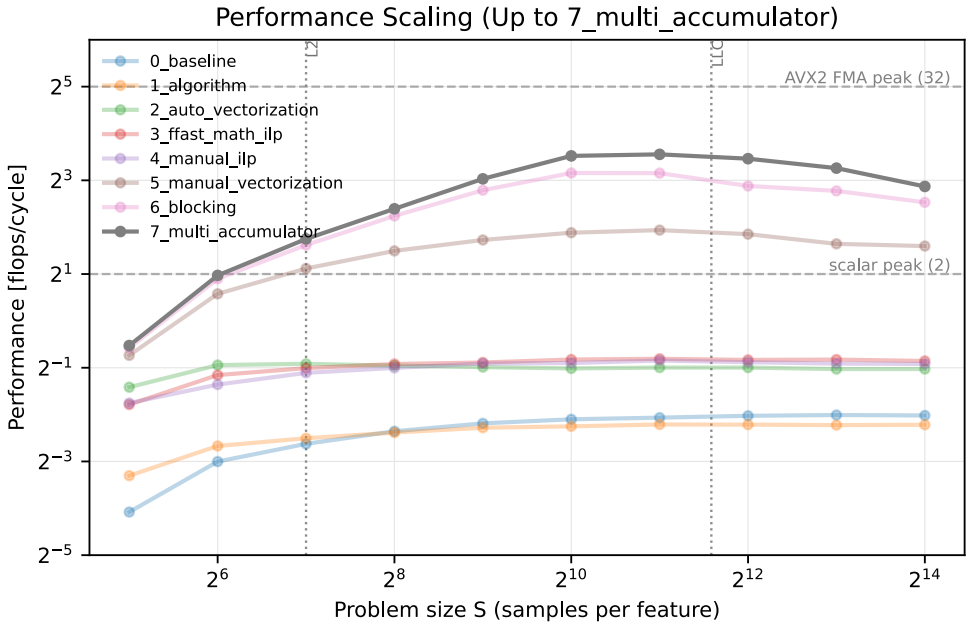
MEETINGS 1 & 2 — RECAP

Incremental optimizations

V0 Baseline	naive reference, `-O3`	0.2×
V1 Logical Baseline	pre-normalize · upper-triangle · min-heap `-O0`	1×
V2 Auto-vectorization	`-O3 -march=native`	2.6×
V3 Fast-math + ILP · V4 Manual ILP	reorder + unroll	~3×
V5 Manual AVX2	8 FMAs/inst	19×
V6 Reg. blocking 4×1 · V7 Multi-acc 4×2	hide FMA lat → ≈ 10 FLOPs/cyc	47×
V8 Sorting	order feat. by interval overlap **new contribution**	53×

cumulative speedup vs logical baseline V1 · `LONG_GAUSSIAN`, $F=2048$, $S=512$

Improving from V7



V7 = 8 accumulators (4×2)
Saturates the 2 FMA ports →
$\approx 10\text{FLOPs/cyc} \approx \frac{1}{3}\pi_{\text{peak}}$

- Inner loop close to optimal (microarch.)
- But touches **every feature pair** and re-walk overlaps blindly
- Dip at $S{=}2^{14}$: $Z = 8\text{MB} > 6\text{MB}$ L3 → DRAM-bound

Untouched lever: the **overlap structure**.
What if we reorder the data to exploit it?

NEW IMPROVEMENT

V8: Sort features by interval

Permute rows → `start[i]` monotone

2 big benefits

Why sorting pays off

1. Monotone starts → early exit

Sorted `start[]` non-decreasing.
For i-block with $E = \max \text{end}[i]$:
$\text{start}[j] \geq E \Rightarrow$ no more overlap
→ `break` loop

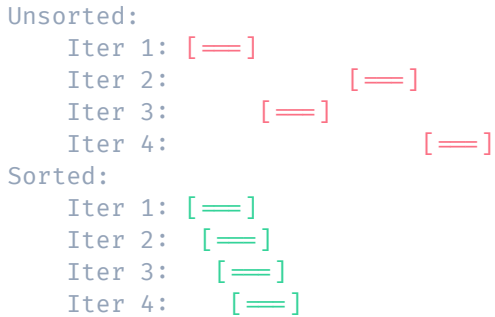
```
uint32_t E = max(end[i..i+3]);
for (j = i+4; ...; j += 2) {
    if (starts[j] >= E) break; // exit
    /* 8-acc 4x2 dot products */
}
```

In `GAUSSIAN_SHORT`, most disjoint → skipped

2: Overlap cluster → cache locality

Adjacent intervals similar $[L, R)$:

- cluster by overlap → strip of Z hot in L1/L2
- `L ... R` tail loop shrink (less redundancy)
- contiguous sorted rows
  → prefetch-friendly strides



V8 permutation implementation

```
// step 1: sort by start
for (i) perm[i] = i;
std::sort(perm, perm+F, [&](a,b){start[a] < start[b]});
for (si) {              // cache sorted bounds
    sorted_starts[si] = starts[perm[si]];
    sorted_ends[si]   = ends[perm[si]];
}
```

```
// step 2: normalize, read original row
//          perm[si], write sorted row si
Z[si*S + t] = (X[perm[si]*S+t] - mean) * inv_std;
```

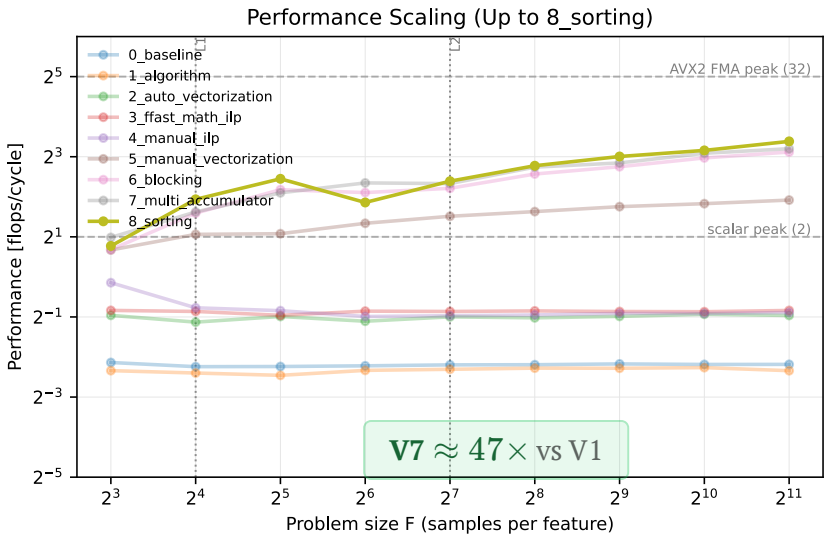
```
// step 3: sorted-space dot-prod: orig. idx → heap
heap_insert(&heaps[i*K], v, perm[j]);
```

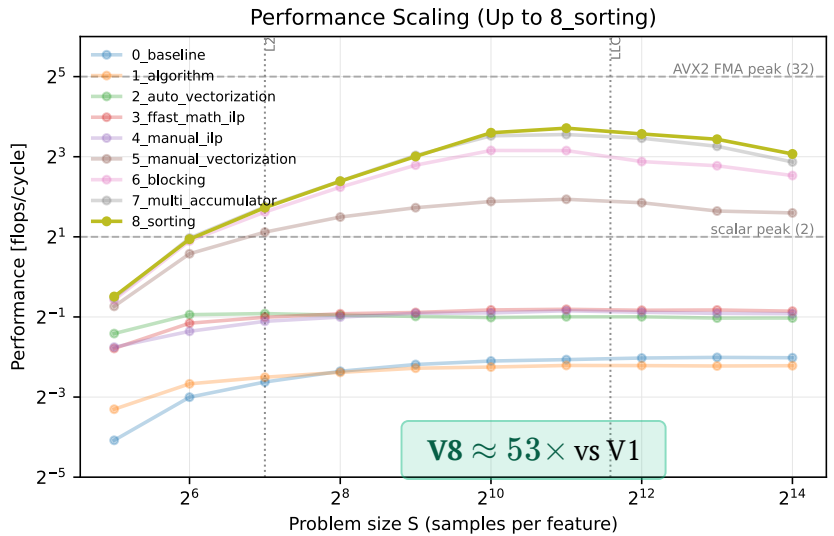
```
// step 4: scatter through perm
heap_to_topK(&heaps[si*K], &topK[perm[si]*K]);
```

Output identical to V7; permutation only internal. Early-exit reduces FLOPs – depends on domain.

`LONG_GAUSSIAN` (default)

Performance vs F (left, $S{=}512$) and vs S (right, $F{=}512$)

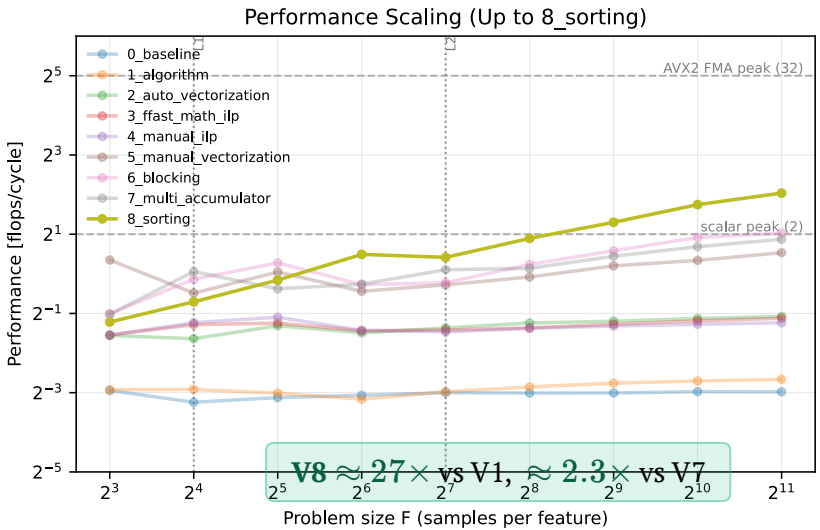


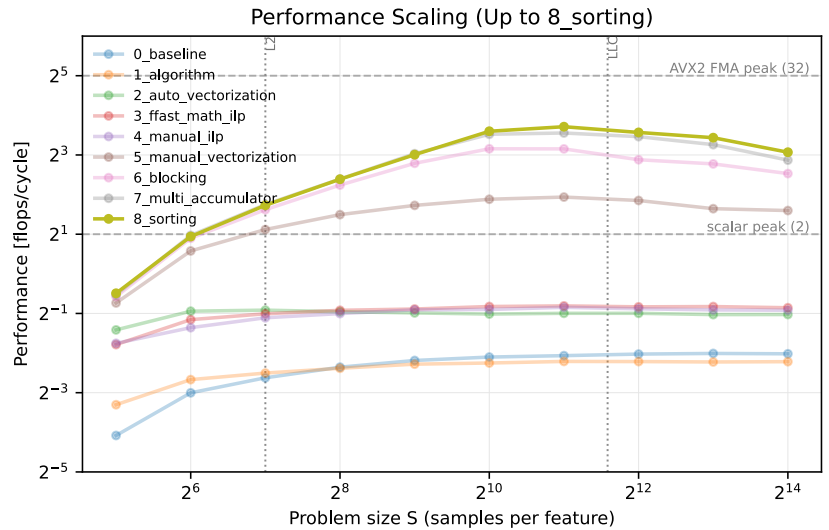


- `LONG` : 80% intervals → most overlap
- Early-exit rare
- Cache locality → still ≈ 5–13% faster at large F

`SHORT_GAUSSIAN` (sorting wins)

Performance: F (left, $S{=}512$) vs S (right, $F{=}512$)

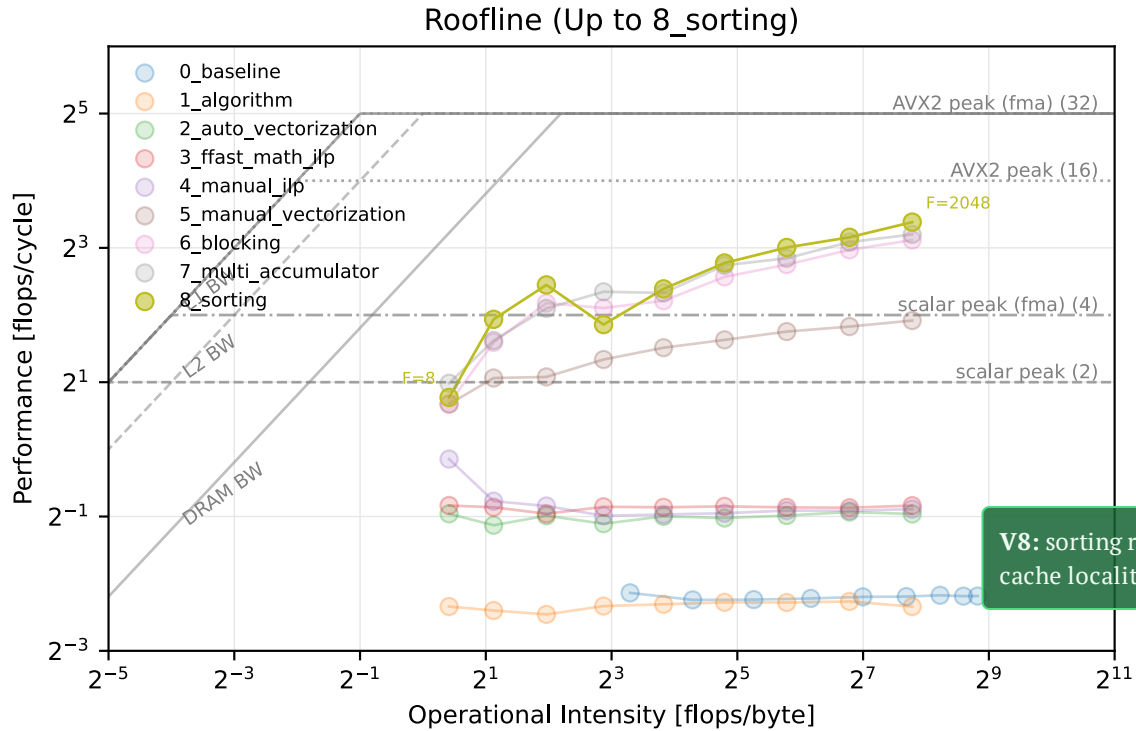




- Reaches $\approx$ 16 FLOPs/cyc
- `SHORT` : 30% intervals $\rightarrow$ mostly disjoint, early-exit prunes
- V7 computes every pair; V8 does not — pure work removed

Rooflines

Operational intensity vs performance, sweep over F $(S{=}512)$



V8: sorting raises block-I-overlap → cache locality

Summary

96.6%

of cycles one dot-product, seen in micro-benchmarks

≈ 10 → 16 FLOPs/cyc

V7 micro-arch peak → V8 sort in `SHORT`

53× / 27×

V8 `LONG` / `SHORT` at $F=2048$ vs logical baseline

- **Meetings 1–2:** micro-architectural improvements (SIMD → blocking → 8 accumulators)
- **New:** data layout improvement – sort by interval
- Regime-dependent: invisible in `LONG`, significant in `SHORT`; adaptivity adheres to R2.

Questions

Team 41

i7-7700HQ (Kaby Lake) · core-pinned, turbo off · PAPI harness · warm branch pred., cold cache